

Initially, I tried to run the simulation environment directly on macOS. However, installing and configuring ROS 2, Gazebo, and Nav2 on a MacBook Air M2 was difficult because of the ARM architecture and compatibility issues. To improve stability and development speed, I installed VMware Fusion Pro and set up an Ubuntu 24.04.4 Desktop ARM64 virtual machine.

After setting up the Ubuntu environment, I installed ROS 2 Jazzy, Gazebo, and Nav2. I first tested the standard ROS 2 Nav2 TurtleBot simulation in Gazebo and RViz to understand how Nav2 performs path planning and obstacle avoidance before applying it to the drone scenario. The TurtleBot demo was useful because it already provides the interfaces expected by Nav2, including odometry, transforms, laser scan data, costmaps, and velocity commands.

In the TurtleBot demo, I initialized the robot position in RViz using “2D Pose Estimate” and then sent navigation goals using “Nav2 Goal.” Nav2 successfully planned paths around obstacles and controlled the robot to reach the selected goals. I also observed that the robot sometimes moved backwards or rotated to avoid obstacles, showing that Nav2 uses its local controller and costmap rather than simply moving directly toward the target.

This confirmed that the Nav2 navigation pipeline works correctly in my Ubuntu VM setup. It also provided a baseline for adapting Nav2 to the drone simulation.

Required Bridges for Applying Nav2 to the Drone Simulation

Nav2 cannot directly control the drones in the current Gazebo setup because the drones are not yet exposed to ROS 2 in the same way as TurtleBot. The current drone simulation is mainly controlled through Gazebo topics and Python scripts, while Nav2 expects ROS 2 topics and coordinate frames.

To apply Nav2 to the drones, several bridges are required. First, Nav2 publishes velocity commands on `/cmd_vel` using `geometry_msgs/msg/Twist`. These commands must be converted into the Gazebo command topic used by the drone. Second, Nav2 needs odometry and transform information, so the drone pose from Gazebo must be converted into `/odom` and `/tf`. Third, Nav2 needs obstacle information, either from a static map, a simulated sensor such as lidar, or directly published obstacle positions.

For the first prototype, I simplified the problem by using only one drone at a fixed altitude. In this setup, Nav2 can be treated as a two-dimensional planner in the horizontal x-y plane, while the drone maintains a constant height.

The intended integration pipeline is:

Gazebo drone pose → ROS 2 `/odom` and `/tf` → Nav2 localization and planning
Nav2 `/cmd_vel` → ROS-Gazebo bridge → Gazebo drone velocity command → drone movement

First Bridge Between ROS 2 and the Gazebo Drone

To begin the integration, I created a new SDF world containing one X3 UAV, a ground plane, and one static obstacle. Using one drone instead of the previous three-drone collision scenario made the first Nav2 integration easier to debug.

The drone model was loaded from the locally downloaded Gazebo Fuel model directory, and the X3 UAV multicopter motor and velocity-control plugins were kept in the SDF. The drone was started at a fixed altitude so that the first prototype could focus on horizontal x-y movement.

I first created a Python bridge script to subscribe to ROS 2 `/cmd_vel`, publish odometry and transform information, and send velocity commands to the drone. However, sending Gazebo commands from Python using repeated `gz topic` subprocess calls caused timeout warnings and was not reliable for continuous control.

To debug this, I inspected the available Gazebo topics for the drone and found the correct command topics:

```
/drone_1/enable  
/drone_1/gazebo/command/motor_speed  
/drone_1/gazebo/command/twist
```

An earlier attempt used `/model/drone_1/gazebo/command/twist`, but this was not the topic used by the drone controller. After identifying the correct topic, I tested direct Gazebo control by publishing `gz.msgs.Twist` messages to `/drone_1/gazebo/command/twist`. The drone moved successfully, confirming that the Gazebo-side controller was working.

Next, I replaced the subprocess-based command method with `ros_gz_bridge`. This bridge converted ROS 2 `geometry_msgs/msg/Twist` messages from `/cmd_vel` into Gazebo `gz.msgs.Twist` messages on `/drone_1/gazebo/command/twist`. After enabling the drone and publishing ROS 2 `/cmd_vel` commands, the drone moved successfully in Gazebo.

This completed the first bridge required for Nav2-drone integration:

```
ROS 2 /cmd_vel → ros_gz_bridge → Gazebo /drone_1/gazebo/command/twist  
→ drone movement
```

This established the command side of the Nav2-drone interface.

Second Bridge: Gazebo Drone Pose to ROS 2 Odometry and TF

After completing the first bridge from ROS 2 `/cmd_vel` to the Gazebo drone velocity command topic, I implemented the second required bridge in the opposite direction. The goal of this bridge was to publish the drone's simulated pose from Gazebo into ROS 2 as `/odom` and `/tf`, so that Nav2 can later know where the drone is located.

For this step, I used Gazebo's world pose information topic. Initially, I tested `/world/drone_nav2_world/pose/info`, but this topic contained pose information for several entities in the world, including the ground plane, obstacle, sun, and drone. I then switched to `/world/drone_nav2_world/dynamic_pose/info`, which is more suitable because it focuses on dynamic entities such as the drone.

The Gazebo dynamic pose topic was bridged into ROS 2 using `ros_gz_bridge` and remapped to `/gz_pose`. I then created a Python node called `drone_pose_to_odom_tf.py`. This node subscribes to `/gz_pose`, extracts the drone pose, and continuously publishes:

```
/odom
odom -> base_link transform
map -> odom static transform
```

During testing, the bridged Gazebo pose messages did not preserve clear frame names, so the script could not directly identify a frame named `drone_1`. To solve this, the Python node was modified to select the most suitable dynamic transform and publish it as the drone pose. Since the current test world contains only one drone, this was sufficient for the single-drone prototype.

After running the dynamic pose bridge and the Python odometry/TF bridge, the `/odom` topic appeared correctly in ROS 2, and `tf2_echo` confirmed that the `odom -> base_link` transform was being published. When the drone was moved using ROS 2 `/cmd_vel`, the odometry values changed accordingly. For example, the x-position increased from approximately `0.0` to values such as `0.27`, `1.97`, and `3.26`, confirming that the ROS 2 odometry was tracking the drone movement.

This completed the second required bridge:

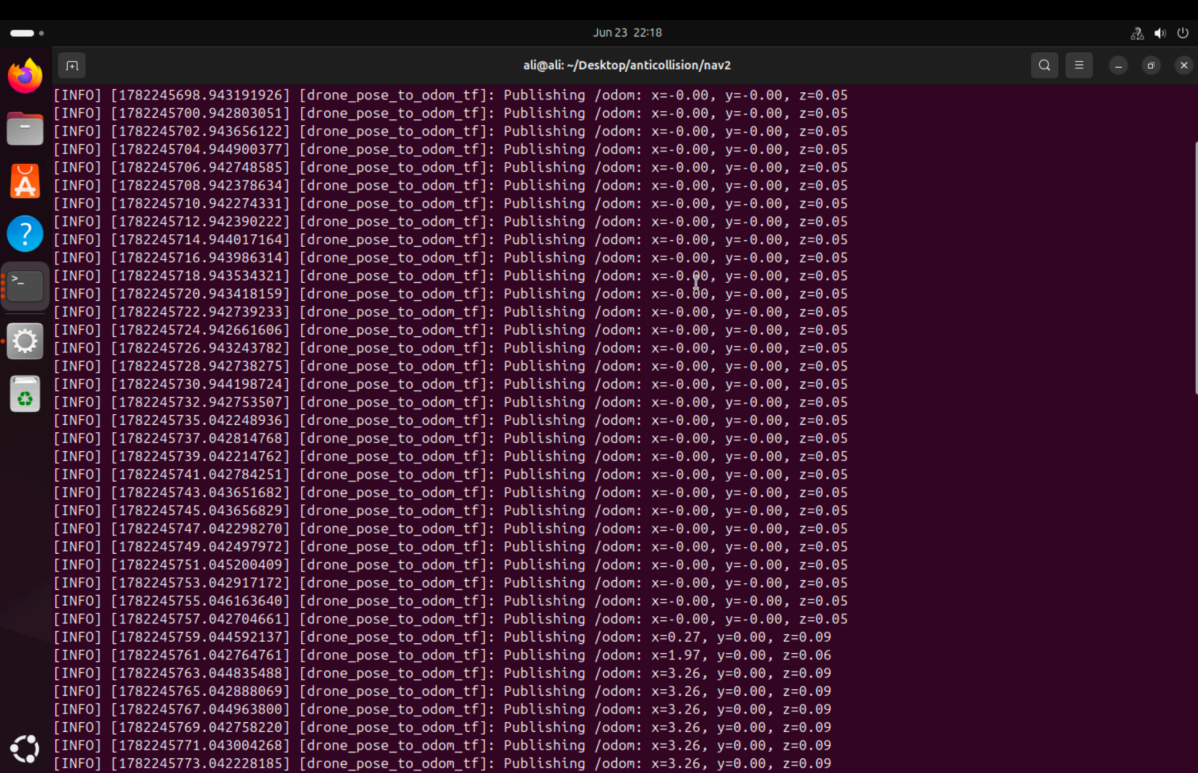
Gazebo drone pose → `ros_gz_bridge` → ROS 2 `/gz_pose` → Python node → `/odom` and `/tf`

At this point, the drone simulation has both basic interfaces needed for future Nav2 integration:

ROS 2 `/cmd_vel` → Gazebo drone movement
Gazebo drone pose → ROS 2 `/odom` and `/tf`

This is an important checkpoint because Nav2 requires both command output and feedback input. The attached log shows that `/odom` is being published correctly and that the ROS

2–Gazebo bridge is working. The next step is to use these two bridges together and test whether Nav2 can send velocity commands to the drone based on its estimated position.

A terminal window titled 'ali@ali: ~/Desktop/anticollision/nav2' showing a series of ROS 2 log messages. The logs are from the 'drone_pose_to_odom_tf' package, indicating that it is publishing /odom data at a rate of 10 Hz. The /odom data is a geometry_msgs/PoseWithCovarianceStamped message. The position (x, y, z) starts at (-0.00, -0.00, 0.05) and remains constant for the first 20 messages. From the 21st message onwards, the x and y coordinates increase to 3.26, while the z coordinate remains at 0.05. The covariance matrix is also shown, with non-zero values for the x and y components. The terminal window has a dark background and a light-colored text color. The left sidebar of the terminal shows various icons for file management and system settings.

```
[INFO] [1782245698.943191926] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245700.942803051] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245702.943656122] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245704.944900377] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245706.942748585] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245708.942378634] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245710.942274331] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245712.942390222] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245714.944017164] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245716.943986314] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245718.943534321] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245720.943418159] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245722.942739233] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245724.942661606] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245726.943243782] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245728.942738275] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245730.944198724] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245732.942753507] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245735.942248936] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245737.942814768] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245739.942214762] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245741.942784251] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245743.943651682] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245745.943656829] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245747.94298270] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245749.942497972] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245751.945200409] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245753.942917172] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245755.946163640] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245757.942704661] [drone_pose_to_odom_tf]: Publishing /odom: x=-0.00, y=-0.00, z=0.05
[INFO] [1782245759.944592137] [drone_pose_to_odom_tf]: Publishing /odom: x=0.27, y=0.00, z=0.09
[INFO] [1782245761.942764761] [drone_pose_to_odom_tf]: Publishing /odom: x=1.97, y=0.00, z=0.06
[INFO] [1782245763.944835488] [drone_pose_to_odom_tf]: Publishing /odom: x=3.26, y=0.00, z=0.09
[INFO] [1782245765.942888069] [drone_pose_to_odom_tf]: Publishing /odom: x=3.26, y=0.00, z=0.09
[INFO] [1782245767.944963800] [drone_pose_to_odom_tf]: Publishing /odom: x=3.26, y=0.00, z=0.09
[INFO] [1782245769.942758220] [drone_pose_to_odom_tf]: Publishing /odom: x=3.26, y=0.00, z=0.09
[INFO] [1782245771.943004268] [drone_pose_to_odom_tf]: Publishing /odom: x=3.26, y=0.00, z=0.09
[INFO] [1782245773.942228185] [drone_pose_to_odom_tf]: Publishing /odom: x=3.26, y=0.00, z=0.09
```

Combined Bridge Test and Automation

After confirming the command bridge and the feedback bridge separately, I tested both bridges together. In this setup, ROS 2 velocity commands are sent to the drone through the `/cmd_vel` bridge, while the drone pose is simultaneously published back to ROS 2 through `/odom` and `/tf`.

To make the testing process easier and more repeatable, I created shell scripts to start and stop the bridge stack automatically. The startup script launches the Gazebo drone world, the Gazebo dynamic pose bridge, the Python odometry/TF bridge, the ROS 2 `/cmd_vel` to Gazebo command bridge, and the drone enable command. I also created a separate movement test script that publishes a forward `/cmd_vel` command for a few seconds and then sends zero velocity commands to stop the drone.

This automation reduces the number of manual terminal commands needed for each test and makes the setup more reliable for repeated experiments. After running the automated stack, the drone moved successfully in Gazebo, and the `/odom` values updated during the movement. This confirmed that the command and feedback bridges can operate together correctly.

The completed automated bridge pipeline is:

`start_drone_stack.sh` → launches Gazebo and all required bridges
`test_move_drone.sh` → publishes `/cmd_vel` and verifies drone movement
`stop_drone_stack.sh` → stops the Gazebo and bridge processes

This step is important because it prepares the system for the next phase, where Nav2 will replace the manual movement test script and generate `/cmd_vel` commands based on navigation goals.

Third Bridge: Obstacle Sensing for Nav2

After the command and odometry bridges, the last missing input was obstacle information. Nav2 builds its costmaps from sensor data. Without a sensor the costmap stays empty and Nav2 plans straight through obstacles. I first added a `gpu_lidar` sensor to the drone in the SDF world, enabled the Gazebo sensors plugin, and bridged the scan topic into ROS 2 as `/scan`.

The sensor registered and its topic appeared in the Gazebo topic list, but it never published any data. The reason is that `gpu_lidar` computes its ranges on the GPU and needs a working OpenGL context, which my VMware virtual machine cannot provide reliably. Without 3D acceleration Gazebo reports "No 3D enabled". With 3D acceleration enabled, the virtual GPU driver is unstable: the Gazebo GUI hangs and the sensor still publishes nothing. Since obstacle information can also come from directly published obstacle positions, I replaced the GPU lidar with a virtual lidar node, `virtual_lidar.py`. It subscribes to `/odom`, casts 360 rays from the drone pose against the known obstacle rectangles, and publishes a normal `sensor_msgs/LaserScan` on `/scan` at 10 Hz. For Nav2 the result looks like a simulated lidar and no rendering is needed. The obstacle list is kept in sync with the world file. On a machine with a working GPU, the real `gpu_lidar` can be used again.

Improved Odometry, Simulation Time, and Cleanup

I replaced the height-based frame selection workaround in the odometry bridge. The drone model now uses Gazebo's `OdometryPublisher` plugin, which publishes the true pose and velocity on its own topic `/model/drone_1/odometry`. The Python bridge subscribes to that topic and republishes `/odom` and the `odom -> base_link` transform. There is no more guessing which entity is the drone, and the velocity comes from the simulator instead of my own finite differences.

I also bridged Gazebo's `/clock` into ROS 2 and enabled `use_sim_time` on all nodes, so everything shares one simulation clock. The old prototype bridge was moved out of the working folder, and every terminal and test run now writes its output into a `logs/` folder. This logging turned out to be very useful for the debugging described below.

Nav2 Configuration and First Obstacle Avoidance Runs

Nav2 runs without a map and without AMCL. The `map -> odom` transform is a static identity, and both costmaps are filled only from `/scan` (an obstacle layer plus an inflation layer). The DWB controller is configured as holonomic so the planner can also use sideways

motion. The standard Jazzy `nav2_bringup` launch file starts extra servers (collision monitor, route server, docking server) that fail without their own configuration, so I wrote a minimal launch file with only the controller, planner, behavior server and BT navigator, plus a lifecycle manager.

The first test scenario is a 1x4x2 m wall at $x = 4$ and a goal at (8, 0), so a straight path would cross the obstacle. The first run succeeded in about 20 seconds with no recovery behaviors. The return flight to (0, 0) aborted after three minutes and 17 recoveries. The logs showed that the drone had actually reached a point 0.24 m from the goal within 20 seconds, which is inside the position tolerance, but it arrived facing away from the requested final heading. Rotating in place produces no translation, so Nav2's progress checker kept reporting "failed to make progress" and finally gave up.

The fix follows from the platform. A quadcopter's final heading does not matter because it can translate in any direction. I disabled the yaw goal tolerance and removed the rotate-to-goal critic from the controller. After this change, repeated back-and-forth runs completed 3 out of 3 with `SUCCEEDED`, about 18 to 19 seconds per 8 m leg, zero recoveries, and a ratio of distance traveled to Euclidean distance of about 1.3. This ratio is the effectiveness metric suggested for the evaluation, computed directly from the recorded logs.

Visualization and Portability

I added two viewers that attach to the running simulation. `view.sh` opens RViz with a preconfigured display: the odometry trail (the actually flown path), the lidar points, the global costmap with its inflation band, and the planned path. Goals can also be sent by clicking in the view. `view_gazebo.sh` attaches Gazebo's own 3D window to the headless server. In the virtual machine the Gazebo GUI only works with 3D acceleration disabled (software rendering is slow but stable). On machines with a real GPU it works normally.

I also removed the remaining portability problems. The world file now loads the X3 model through its Gazebo Fuel URI instead of a local absolute path, all scripts find their own directory instead of assuming a fixed one, and the repository README documents the requirements and the step-by-step procedure. The project now runs unchanged on other machines.

Phase A: Random Scenarios and Hit Counting

For the testing protocol I wrote a scenario generator, `gen_scenario.py`. It takes the number of obstacles and a random seed, then picks a random start point, a random goal point at least 8 m away, and N random box obstacles with obstacle-free zones around the start and the goal. It writes both the Gazebo world (`world.sdf`, with the drone spawned at the random start) and a machine-readable `scenario.yaml` from the same data, so the simulation, the virtual lidar and the collision counter always agree about the obstacle positions. The same seed always produces the same scenario.

Collision counting follows the protocol: the run is not stopped on contact. A new node, `hit_monitor.py`, models the drone as a 0.3 m disc, counts each contact episode with an obstacle, and tracks the minimum clearance over the whole flight. Hits are counted from the moment the goal is sent until it finishes. Contacts that happen while the drone is parked (for

example an obstacle drifting over it before the goal is sent) are excluded, since they are not navigation failures.

The first randomized runs exposed one more Nav2 default that does not fit a multicopter. Goals that required mostly sideways flight stalled, while forward goals worked. Recording `/cmd_vel` showed that Nav2 was sending correct velocity commands and that the drone ignored them. By replaying the commands manually I found the cause: any command with a sustained yaw rate stalls the X3 velocity controller completely, while pure translation works. I therefore removed the two heading alignment critics and set the controller to command no rotation at all (`max_vel_theta: 0.0`). The drone keeps its spawn heading and moves by strafing, which a quadcopter can do freely. After this fix the verification scenarios passed: 4 obstacles with a 12 m sideways path (**SUCCEEDED** in 32 s, no recoveries, no hits) and a 10-obstacle scenario (**SUCCEEDED** in 18 s, no hits, minimum clearance 0.59 m).

How to Run (Current Procedure)

Each numbered step runs in its own terminal from the `nav2/` folder:

`./gen_scenario.py --n-obstacles 4 --seed 42` generates a scenario (into `scenarios/s42_n4/`).

`./start_drone_stack.sh scenarios/s42_n4` starts Gazebo (headless), the bridges, the virtual lidar, the hit monitor and the obstacle mover. Wait until the odometry terminal prints **Publishing /odom** lines. Without the argument, the classic single-wall world is used.

`./launch_nav2.sh` starts Nav2. Wait for "Managed nodes are active".

`./view.sh` (optional) opens the RViz view: costmap, planned path, lidar points, odometry trail and the obstacle markers.

`./view_gazebo.sh` (optional) attaches the Gazebo 3D window. In the VM this requires 3D acceleration to be disabled.

`./send_nav2_goal.sh --scenario scenarios/s42_n4` arms the drone and sends the scenario goal. `./send_nav2_goal.sh X Y` sends an explicit goal instead.

`cat logs/hits_current.yaml` shows hits and minimum clearance. The full feedback stream of every goal is kept in `logs/goal_<time>.txt`.

`./stop_drone_stack.sh` stops everything (stop Nav2 with Ctrl+C in its terminal).

Phase B: Batch Evaluation with Static Obstacles

To produce the statistics I wrote a batch runner, `run_batch.sh`. For every combination of obstacle count and seed it generates a scenario, starts the whole stack in the background (without terminal windows), waits until odometry and Nav2 are ready, arms the drone, sends the goal with a watchdog timeout, saves all logs and results, and shuts everything down before the next run. If a run fails, the batch records the failure and continues. A second script, `compute_metrics.py`, reads the recorded files and computes success rate, navigation time, the path to Euclidean distance ratio, hit count and minimum clearance per obstacle count, and creates the corresponding plots.

The main experiment swept 1, 2, 4, 6, 8 and 10 obstacles with 8 random seeds each, 48 runs in about 46 minutes without supervision. All 48 runs reached the goal with zero hits:

N	success	mean time	path/Euclid	hits	min clearance
1	8/8	29.3 s	0.980	0	0.80 m
2	8/8	29.3 s	0.981	0	0.76 m
4	8/8	29.3 s	0.986	0	0.52 m
6	8/8	30.6 s	1.001	0	0.38 m
8	8/8	31.5 s	1.019	0	0.37 m
10	8/8	31.6 s	1.021	0	0.37 m

The trends are as expected. With few obstacles the direct line is usually free, so the ratio is slightly below 1 (the drone stops 0.3 m before the goal point, inside the goal tolerance). With more obstacles the navigation time and the detour grow, and the minimum clearance drops toward the costmap inflation radius, which acts as the safety margin.

In the first attempts a few runs failed with timeout errors inside Nav2. The cause was the limited RAM of the virtual machine: when the VM ran out of memory, the simulation froze for a while and Nav2's internal service calls timed out. These were infrastructure failures, not navigation failures. I increased a Nav2 timeout parameter (`default_server_timeout`) and repeated the affected runs with the same seeds and the same configuration, which gave the final 48 out of 48. The curated results (CSV, summary table, plots) are stored in the repository under `results/2026-07-17_static_sweep/`.

Phase C: Moving Obstacles

The last part of the protocol repeats the evaluation with moving obstacles. The motion is implemented in the same virtual way as the sensing. A new node, `obstacle_mover.py`, moves the obstacle rectangles at a constant speed in random directions and lets them bounce off the workspace bounds. It publishes the current positions at 20 Hz, and the virtual lidar, the hit monitor and RViz all use this single source, so the drone always senses the obstacles where they currently are. Moving obstacles are not placed in the Gazebo world, because a physical box frozen at its spawn position could collide with the drone at a place where the obstacle virtually no longer is.

The sweep used 6 obstacles, speeds from 0.2 to 2.0 m/s, and 8 seeds per speed (48 runs):

v (m/s)	success	mean time	path/Euclid	hits
0.2	8/8	32 s	1.11	1
0.4	8/8	39 s	1.15	19
0.8	8/8	48 s	1.38	9
1.2	8/8	50 s	1.48	35
1.6	8/8	71 s	1.97	50
2.0	8/8	71 s	1.81	73

Because the runs are not stopped on contact, the drone always reaches the goal in the end, so the success column stays at 8/8. The degradation shows in the other columns. The hit count grows from 1 contact in eight flights at 0.2 m/s to 73 contacts (about nine per flight) at 2.0 m/s. The navigation time roughly doubles, and the flown path grows to about twice the straight distance. The explanation is that Nav2 only reacts to where obstacles currently are and knows nothing about their velocity. The drone's own speed limit is about 0.6 to 0.7 m/s, so from roughly 0.8 m/s the obstacles are faster than the drone and reactive replanning cannot keep up.

This completes the agreed evaluation. Nav2 handles static and slowly moving obstacles well, and its performance deteriorates significantly once the obstacle speed approaches the drone's own speed. This is the baseline that motivates the avoidance component we want to add on top of Nav2. The same scenario generator, batch runner and metrics can be reused later for the comparison with and without the component. The curated results are in [results/2026-07-20_speed_sweep/](#).